

Assignment

1. Follow the prescribed procedure to identify compiler errors in the given C program by carefully examining each statement, detecting all errors, classifying them as lexical, syntax, semantic errors, logical errors or runtime errors and rewriting the incorrect statements with their appropriate corrections.

```
#include <stdio.h>
```

```
int calculator()
{
    int 1a = 10, b =20;
    float d = 2.5;
    int x;
    x = 1a + d;
    int y = b - 20
    int z = b / y ;
    if ( 1a < b )
        small = b ;
    print("%d is smaller" b )
    return d;
}
```

```
int main()
{
    calculator();
    return 0;
}
```

2. Follow the prescribed procedure to identify the common prefixes in the following grammar, apply the left factoring technique step by step, and write the equivalent left-factored grammar.

```
S → aAB | aAC | aD | bA
A → cde | cdf | cdg | h
B → ij | ik
C → il | im
D → cde | cdh
```

3. Follow the prescribed procedure to construct the recursive descent parsing functions for the given grammar and demonstrate step-by-step parsing of the input string using those function. **Input string:** a a b b c d

```
S → A B C
A → a A | a
B → b B | b | ε
C → c C | d
```

4. Follow the prescribed procedure to identify lexemes, corresponding patterns, and tokens from the given source code by scanning each statement and classifying every valid lexeme into its correct token category.

```
#include <stdio.h>

int main()
{
    int num = 25;
    float value = 5.5;
    if (num > value)
        printf("Greater value");
    return 0;
}
```

5. Follow the prescribed procedure to determine whether the given grammar is LL(1) by eliminating left recursion (if any), performing left factoring (if required), computing FIRST and FOLLOW sets, and constructing the LL(1) parsing table.

```
S → S A B c | E f
A → a A | ε
B → b B | b d
E → a | ε
```